



# ΣΥΣΤΥΜΑΤΑ ΕΥΦΥΩΝ ΠΡΑΚΤΟΡΩΝ, 2<sup>η</sup> εργασία

| ΟΝΟΜΑΤΕΠΩΝΥΜΟ | ΑΡΙΘΜΟΣ<br>ΜΗΤΡΩΟΥ | EMAIL                   | ΗΜΕΡΟΜΗΝΙΑ |
|---------------|--------------------|-------------------------|------------|
| ELVIS AVDULI  | E22001             | avdulielvis@gmail.com   | 11/01/2026 |
| ΚΑΛΟΥΣ ΜΑΡΙΟΣ | E22066             | kalousmarios3@gmail.com |            |

**7<sup>ο</sup> Εξάμηνο**



## Πίνακας περιεχομένων

|                                                          |    |
|----------------------------------------------------------|----|
| 1. Εισαγωγή .....                                        | 3  |
| 2. Αλλαγές από το Μονοπρακτορικό Σύστημα .....           | 3  |
| 2.1 Χώρος Καταστάσεων .....                              | 3  |
| 2.2 Νέοι Περιορισμοί .....                               | 3  |
| 3. Πρωτόκολλο Διαπραγμάτευσης .....                      | 4  |
| 3.1 Αρχιτεκτονική Πρωτοκόλλου .....                      | 4  |
| 3.2 Υπολογισμός Χρησιμότητας (Utility Calculation) ..... | 4  |
| 3.3 Διαδικασία Διαπραγμάτευσης .....                     | 6  |
| 3.4 Πλεονεκτήματα του Πρωτοκόλλου .....                  | 6  |
| 4. Αποφυγή Συγκρούσεων .....                             | 7  |
| 4.1 Χωρικές Συγκρούσεις (Spatial Conflicts) .....        | 7  |
| 4.2 Στρατηγική Retreat & Wait .....                      | 8  |
| 4.3 Συγκρούσεις στα Αντικείμενα .....                    | 9  |
| 4.4 Oscillation Detection .....                          | 9  |
| 5. Διαχείριση Αποτυχιών .....                            | 10 |
| 5.1 Ανίχνευση Αποτυχιών .....                            | 10 |
| 5.2 Ιεραρχία Recovery Strategies .....                   | 10 |
| 5.3 Παράδειγμα Σεναρίου Αποτυχίας .....                  | 11 |
| 6. Πειραματική Μελέτη - Στατικό Περιβάλλον .....         | 12 |
| 6.1 Μεθοδολογία .....                                    | 12 |
| 6.2 Συγκεντρωτικά Αποτελέσματα .....                     | 12 |
| 6.3 Κατανομή Utilities .....                             | 13 |
| 6.4 TOP 5 Runs - Ανάλυση Επιτυχίας .....                 | 13 |
| 6.5 Κατανομή Εργασιών μεταξύ Πρακτόρων .....             | 14 |
| 6.6 Συσχέτιση Utilities .....                            | 15 |
| 6.7 Ανάλυση Βημάτων .....                                | 15 |
| 6.8 Collision Events .....                               | 16 |
| 7. Δυναμικό Περιβάλλον - Συνοπτική Ανάλυση .....         | 17 |
| 7.1 Επίδραση της Δυναμικότητας .....                     | 17 |



|                                                |    |
|------------------------------------------------|----|
| 7.2 Μηχανισμός Replanning .....                | 17 |
| 7.3 Worst Case Scenarios .....                 | 18 |
| 7.4 Συσχέτιση στο Δυναμικό .....               | 18 |
| 8. Περιπτώσεις Αποτυχίας του Πρωτοκόλλου ..... | 18 |
| 8.1 Circular Dependency Deadlock .....         | 18 |
| 8.2 Corridor Blocking .....                    | 19 |
| 8.3 Race Condition στα Αντικείμενα .....       | 19 |
| 8.4 Oscillation Loop .....                     | 20 |
| 8.5 Γενικές Παρατηρήσεις .....                 | 20 |
| 9. Συμπεράσματα .....                          | 21 |

# 1. Εισαγωγή

Η παρούσα εργασία επεκτείνει το μονοπρακτορικό σύστημα της 1ης άσκησης σε ένα **πολυπρακτορικό περιβάλλον** με δύο πράκτορες που συνεργάζονται για την επίτευξη κοινών στόχων. Η βασική πρόκληση είναι η **αποφυγή συγκρούσεων** τόσο στην ανάθεση εργασιών όσο και στην κίνηση στο πλέγμα, μέσω ενός **πρωτοκόλλου διαπραγμάτευσης** βασισμένου σε χρησιμότητα (utility-based negotiation).

## Νέα Χαρακτηριστικά του Συστήματος

- **Δύο πράκτορες:** Agent\_1 (θέση 1,1) και Agent\_2 (θέση 3,3)
- **Πρωτόκολλο διαπραγμάτευσης** για ανάθεση εργασιών
- **Αποφυγή χωρικών συγκρούσεων** (collision avoidance)
- **Διαχείριση κοινών πόρων** (αντικείμενα)
- **Ανίχνευση και επίλυση αδιεξόδων** (deadlock detection & resolution)
- **Γραφικό περιβάλλον** για καλύτερη μελέτη της συμπεριφοράς των πρακτόρων

# 2. Αλλαγές από το Μονοπρακτορικό Σύστημα

## 2.1 Χώρος Καταστάσεων

Ο χώρος καταστάσεων επεκτείνεται για να περιλάβει δύο πράκτορες:

- Θέσεις:  $21 \times 21 = 441$  δυνατοί συνδυασμοί θέσεων
  - Inventory:  $15 \times 15 = 225$  δυνατοί συνδυασμοί αντικειμένων
  - Στόχοι: 8 καταστάσεις
  - Θέσεις στόχων: 7980 διατάξεις
- Συνολικός χώρος:  $441 \times 225 \times 8 \times 7980 \approx 6.3 \times 10^9$  καταστάσεις

## 2.2 Νέοι Περιορισμοί

1. **Χωρική Αποκλειστικότητα:** Οι πράκτορες δεν μπορούν να καταλάβουν το ίδιο κελί



2. **Αποκλειστικότητα Αντικειμένων:** Ένα αντικείμενο μπορεί να το κατέχει μόνο ένας πράκτορας
3. **Αποκλειστικότητα Εργασιών:** Κάθε εργασία εκτελείται από έναν πράκτορα

### 3. Πρωτόκολλο Διαπραγμάτευσης

#### 3.1 Αρχιτεκτονική Πρωτοκόλλου

Το πρωτόκολλο ακολουθεί το **Contract Net Protocol** με τροποποιήσεις:

##### ΦΑΣΗ 1: ΥΠΟΛΟΓΙΣΜΟΣ ΧΡΗΣΙΜΟΤΗΤΑΣ

- Κάθε πράκτορας υπολογίζει utility για κάθε διαθέσιμη εργασία
- Λαμβάνει υπόψη: απόσταση, διαθέσιμα items, ανταμοιβή
- Ελέγχει αν ο άλλος πράκτορας έχει απαραίτητα items

##### ΦΑΣΗ 2: ΔΗΜΟΣΙΕΥΣΗ ΠΡΟΤΑΣΗΣ

- -Broadcast: proposal(Task, AgentName, Utility)
- -Χρόνος αναμονής: 500ms
- -Συλλογή όλων των proposals

##### ΦΑΣΗ 3: ΑΞΙΟΛΟΓΗΣΗ & ΑΠΟΦΑΣΗ

- Σύγκριση utilities από όλους τους πράκτορες
- Tie-breaking: Προτεραιότητα στον Agent\_1
- Ανακοίνωση νικητή

##### ΦΑΣΗ 4: ΕΚΤΕΛΕΣΗ

- Νικητής: Εκτελεί την εργασία
- Ηττημένος: Αναζητά εναλλακτική εργασία ή wander

#### 3.2 Υπολογισμός Χρησιμότητας (Utility Calculation)

##### Java Implementation (UtilityService.java)

```
private double calculateTaskUtility(int currentX, int currentY,  
                                   int goalX, int goalY,  
                                   double reward,  
                                   boolean hasItem1, boolean hasItem2) {  
  
    int goalDistance = PathFindingService.getPathLength(currentX, currentY,
```



```
goalX, goalY);
```

```
if (goalDistance == 0) goalDistance = 1; // Avoid division by zero
```

```
// Αν έχουμε και τα 2 items → μόνο η απόσταση μετράει
```

```
if (hasItem1 && hasItem2) {  
    return reward / goalDistance;  
}
```

```
// Αλλιώς εκτιμούμε το κόστος συλλογής
```

```
int collectionCost = 0;
```

```
if (!hasItem1) collectionCost += 3; // Μέση απόσταση
```

```
if (!hasItem2) collectionCost += 3;
```

```
int totalDistance = goalDistance + collectionCost;
```

```
return totalDistance > 0 ? reward / totalDistance : 0.0;
```

```
}
```

#### Jason Implementation (agent.asl)

```
% Utility = 0 αν ο άλλος πράκτορας έχει απαραίτητα items
```

```
+!estimate_utility(table, 0) :
```

```
    other_agent_has(brush) | other_agent_has(color).
```

```
% Υψηλό utility αν έχουμε όλα τα items
```

```
+!estimate_utility(table, Utility) :
```

```
    position(X, Y) &
```

```
    table_at(TX, TY) &
```

```
    carrying_item(brush) &
```

```
    carrying_item(color)
```

```
<- Dist = math.abs(TX - X) + math.abs(TY - Y);
```

```
    Utility = 1.0 / (Dist + 1).
```

```
% Μέτριο utility αν χρειάζονται items
```

```
+!estimate_utility(table, Utility) :
```

```
    position(X, Y) &
```

```
    table_at(TX, TY)
```

```
<- Dist = math.abs(TX - X) + math.abs(TY - Y);
```

```
    Utility = 1.0 / (Dist + 7). // Penalty για συλλογή
```

### 3.3 Διαδικασία Διαπραγμάτευσης

Η διαπραγμάτευση λειτουργεί ως **auction-based system** όπου κάθε πράκτορας "προσφέρει" για την εργασία με το utility του ως bid.

**Βήμα 1 - Έναρξη:** Όταν ένας πράκτορας είναι έτοιμος για νέα εργασία, υπολογίζει utility για όλες τις διαθέσιμες εργασίες και εκδηλώνει ενδιαφέρον για αυτή με το υψηλότερο utility.

**Βήμα 2 - Broadcast:** Ο πράκτορας στέλνει μήνυμα proposal(Task, AgentName, Utility) σε όλους. Το μήνυμα περιέχει την εργασία που θέλει, την ταυτότητά του και το υπολογισμένο utility.

**Βήμα 3 - Αναμονή:** Οι πράκτορες περιμένουν 500ms για να συλλέξουν όλες τις προτάσεις. Αυτό το time window εξασφαλίζει ότι όλοι έχουν την ευκαιρία να εκφράσουν το ενδιαφέρον τους.

**Βήμα 4 - Σύγκριση:** Κάθε πράκτορας συγκρίνει το δικό του utility με αυτά των άλλων για την ίδια εργασία. Ο πράκτορας με το υψηλότερο utility κερδίζει την εργασία.

**Βήμα 5 - Tie-breaking:** Σε περίπτωση ισοπαλίας (ίδιο utility), δίνεται προτεραιότητα στον Agent\_1. Αυτή η απόφαση εξασφαλίζει **ντετερμινισμό** και αποφεύγει ατέρμονες επαναδιαπραγματεύσεις.

**Βήμα 6 - Εκτέλεση:** Ο νικητής προχωρά στην εκτέλεση της εργασίας. Ο ηττημένος αναζητά άλλη διαθέσιμη εργασία ή εκτελεί wander αν δεν υπάρχει κάτι άλλο.

### 3.4 Πλεονεκτήματα του Πρωτοκόλλου

**Αποδοτικότητα:** Κάθε εργασία ανατίθεται στον πράκτορα που μπορεί να την ολοκληρώσει με το μικρότερο κόστος (βήματα).

**Αποφυγή Συγκρούσεων:** Μόνο ένας πράκτορας αναλαμβάνει κάθε εργασία, αποφεύγοντας διπλές προσπάθειες.

**Αυτονομία:** Κάθε πράκτορας λαμβάνει αποφάσεις ανεξάρτητα με βάση την τοπική του γνώση.

**Προσαρμοστικότητα:** Αν αλλάξουν οι συνθήκες (π.χ. ένας πράκτορας πάρει item), το utility ανανεώνεται στην επόμενη διαπραγμάτευση.

## 4. Αποφυγή Συγκρούσεων

### 4.1 Χωρικές Συγκρούσεις (Spatial Conflicts)

Η χωρική σύγκρουση είναι το πιο σύνηθες πρόβλημα στο πολυπρακτορικό σύστημα. Οι δύο πράκτορες, ενώ εκτελούν τις εργασίες τους, μπορεί να βρεθούν να εμποδίζουν ο ένας τον άλλον.

#### Ανίχνευση Blocking

Το σύστημα ανιχνεύει blocking όταν:

1. Ένας πράκτορας προσπαθεί να κινηθεί αλλά αποτυγχάνει (το επόμενο κελί είναι κατειλημμένο)
2. Η αποτυχία επαναλαμβάνεται 3 συνεχόμενες φορές στην ίδια θέση
3. Ο πράκτορας ταλαντώνεται μεταξύ 2 θέσεων (oscillation)

Όταν ανιχνευτεί blocking, το σύστημα εκτελεί διάγνωση για να καθορίσει ποιος πράκτορας εμποδίζει ποιον. Αυτό γίνεται με υπολογισμό του επόμενου βήματος στο path του κάθε πράκτορα και έλεγχο αν αυτό το κελί είναι κατειλημμένο από τον άλλον.

#### Στρατηγική Move-Aside

Όταν ένας πράκτορας (A) ανιχνεύσει ότι ο άλλος πράκτορας (B) τον εμποδίζει, στέλνει μήνυμα `please_move_aside(A, X, Y)` όπου (X,Y) είναι η θέση που καταλαμβάνει ο B.

Ο B, όταν λάβει το μήνυμα και επαληθεύσει ότι είναι πράγματι στη θέση (X,Y), εκτελεί `move-aside maneuver`:

Κάθετη Απομάκρυνση (Προτεραιότητα 1): Αν ο B και ο A είναι στην ίδια στήλη ή σειρά, ο B κινείται κάθετα για να δώσει χώρο. Για παράδειγμα:

- Αν είναι στην ίδια στήλη (π.χ. A στο (2,2) και B στο (2,3)), ο B κινείται αριστερά ή δεξιά
- Αν είναι στην ίδια σειρά (π.χ. A στο (2,2) και B στο (3,2)), ο B κινείται πάνω ή κάτω

Αυτή η στρατηγική είναι αποδοτική σε στενούς διαδρόμους όπου η κίνηση μακριά από τον A μπορεί να οδηγήσει σε αδιέξοδο.





Διαγώνια Απομάκρυνση (Προτεραιότητα 2): Αν δεν υπάρχει κάθετος χώρος, ο B κινείται διαγώνια μακριά από τον A. Ο προσανατολισμός υπολογίζεται ως:

$$DX = B.x - A.x$$

$$DY = B.y - A.y$$

→ Κινείται προς την κατεύθυνση (DX, DY)

Ο B εκτελεί 3 βήματα move-aside για να εξασφαλίσει επαρκή απομάκρυνση. Μετά από κάθε βήμα, περιμένει 250ms για να δώσει χρόνο στον A να προχωρήσει.

### Επαλήθευση Κίνησης

Μετά την αποστολή του μηνύματος move-aside, ο A περιμένει 1000ms και στη συνέχεια επαληθεύει αν ο B όντως κινήθηκε. Αυτό γίνεται συγκρίνοντας τη θέση του B πριν και μετά το μήνυμα.

Αν ο B δεν κινήθηκε (είτε γιατί είναι περικυκλωμένος από εμπόδια, είτε γιατί το μήνυμα χάθηκε), ο A εκτελεί retreat strategy - κινείται προς τα πίσω για να δώσει χώρο και επαναπροσπαθεί αργότερα.

## 4.2 Στρατηγική Retreat & Wait

Όταν η move-aside στρατηγική αποτύχει (ο blocking πράκτορας δεν μπορεί να κινήθει), ο πράκτορας που περιμένει εκτελεί tactical retreat:

1. Κινείται 4 βήματα προς τα πίσω (προς την προηγούμενή του θέση)
2. Περιμένει 1500ms για να δώσει χρόνο στον άλλον πράκτορα να ολοκληρώσει τη δική του κίνηση
3. Ξαναπροσπαθεί την πλοήγηση

Αν η retreat αποτύχει 3 φορές ( $retreat\_count \geq 3$ ), ο πράκτορας εκτελεί full abort:

- Εγκαταλείπει εντελώς την τρέχουσα εργασία
- Μετακινείται σε "ασφαλή ζώνη" (safe zone) στο πλέγμα
- Περιμένει 2000ms
- Επιστρέφει για νέα διαπραγμάτευση

Οι ασφαλείς ζώνες είναι:

- Για πράκτορες στην κάτω-αριστερή περιοχή ( $X \leq 2, Y \leq 2$ ): Μετακίνηση στο (5,5)
- Για πράκτορες στην πάνω-δεξιά περιοχή ( $X \geq 4, Y \geq 4$ ): Μετακίνηση στο (1,1)
- Διαφορετικά: Μετακίνηση στην απέναντι πλευρά του πλέγματος

Αυτή η στρατηγική εξασφαλίζει χωρικό διαχωρισμό όταν οι τοπικές στρατηγικές αποτύχουν.

### 4.3 Συγκρούσεις στα Αντικείμενα

Η σύγκρουση στα αντικείμενα συμβαίνει όταν και οι δύο πράκτορες θέλουν το ίδιο item. Το σύστημα αντιμετωπίζει αυτό σε τρία επίπεδα:

**1. Πρόληψη μέσω Utility:** Αν ένας πράκτορας έχει ήδη ένα item, το utility του άλλου πράκτορα για εργασίες που χρειάζονται αυτό το item μηδενίζεται. Έτσι, η διαπραγμάτευση φυσικά αποφεύγει την ανάθεση τέτοιων εργασιών.

**2. Άρνηση Pickup:** Πριν προσπαθήσει να πάρει ένα item, ο πράκτορας ελέγχει αν το έχει ήδη ο άλλος (`other_agent_has(Item)`). Αν ναι, η ενέργεια pickup ακυρώνεται άμεσα.

**3. Race Condition Handling:** Στη σπάνια περίπτωση που και οι δύο πράκτορες φτάσουν ταυτόχρονα στο ίδιο item, το περιβάλλον επιτρέπει το pickup μόνο στον πρώτο που το ζήτησε. Ο δεύτερος λαμβάνει αποτυχία και επανεκτιμά τις επιλογές του.

### 4.4 Oscillation Detection

Το oscillation συμβαίνει όταν ένας πράκτορας κινείται επανειλημμένα μεταξύ δύο θέσεων χωρίς πρόοδο. Το σύστημα ανιχνεύει αυτό με:

1. **Ιστορικό Θέσεων:** Κρατάει buffer των τελευταίων 10 θέσεων κάθε πράκτορα
2. **Pattern Matching:** Ελέγχει αν το buffer περιέχει μόνο 2 μοναδικές θέσεις
3. **Counter:** Αν η τρέχουσα θέση == προηγούμενη θέση, αυξάνει `oscillation_count`

Όταν `oscillation_count`  $\geq 2$ , ενεργοποιείται **pattern breaking**:

- Ο Agent\_1 μετακινείται σε safe zone και περιμένει 1500ms (passive strategy)

- Ο Agent\_2 εκτελεί active clear\_area (800ms wait)

Η ασύμμετρη αντιμετώπιση αποφεύγει την περίπτωση και οι δύο να ταλαντώνονται συγχρονισμένα.

## 5. Διαχείριση Αποτυχιών

Το σύστημα παρακολουθεί αποτυχίες σε δύο επίπεδα: **επίπεδο ενέργειας** (action failures) και **επίπεδο εργασίας** (task failures).

### 5.1 Ανίχνευση Αποτυχιών

Μια εργασία θεωρείται αποτυχημένη όταν:

- Ο πράκτορας δεν μπορεί να συλλέξει τα απαιτούμενα items (ο άλλος τα έχει)
- Η πλοήγηση αποτυγχάνει επανειλημμένα (stuck\_count  $\geq 3$ )
- Η εργασία δεν ολοκληρώνεται μετά από 5 deadlock cycles

Κάθε αποτυχία καταγράφεται στο fail\_count και η εργασία επισημαίνεται ως failed\_task(Task).

### 5.2 Ιεραρχία Recovery Strategies

**Level 1 - Simple Recovery** (fail\_count = 1):

1. Απόθεση όλων των items (drop\_all\_items)
2. Εκτέλεση wander για 1000ms
3. Νέα διαπραγμάτευση

Αυτή η στρατηγική αντιμετωπίζει περιπτώσεις όπου ο πράκτορας έχει λάθος items ή η εργασία δεν είναι πλέον εφικτή λόγω του άλλου πράκτορα.

**Level 2 - Moderate Recovery** (fail\_count = 2):

1. Drop items
2. Clear area (απομάκρυνση από τρέχουσα περιοχή)
3. Extended wander (1500ms)
4. Νέα διαπραγμάτευση



Αυτή η στρατηγική προσθέτει χωρικό διαχωρισμό για να αποφύγει επαναλαμβανόμενες συγκρούσεις στην ίδια περιοχή.

**Level 3 - Critical Recovery** ( $\text{fail\_count} \geq 3$ ):

1. **Force drop** όλων των items
2. **Full clear area** με aggressive movement
3. **Extended timeout** 2000ms
4. **Reset counters**:  $\text{failed\_task} = \text{none}$ ,  $\text{fail\_count} = 0$
5. **Fresh start** με νέα διαπραγμάτευση

Αυτή η στρατηγική αποτελεί "hard reset" για να διασπάσει παγιωμένες καταστάσεις (π.χ. και οι δύο πράκτορες σε cyclic dependencies).

## 5.3 Παράδειγμα Σεναρίου Αποτυχίας

**Κατάσταση:**

- Agent\_1 θέλει να βάψει table (χρειάζεται brush, color)
- Agent\_2 έχει ήδη brush και color και βάφει chair
- Agent\_1 προσπαθεί να πάρει brush → Αποτυχία (το έχει Agent\_2)
- Agent\_1 δοκιμάζει να βρει color → Αποτυχία (το έχει Agent\_2)

**Recovery Flow:**

1. Πρώτη αποτυχία: Drop items (αν έχει), wander, προσπαθεί ξανά
2. Δεύτερη αποτυχία: Απομακρύνεται από την περιοχή, περιμένει περισσότερο
3. Τρίτη αποτυχία: **Critical recovery** - reset και αναζήτηση εντελώς νέας εργασίας (π.χ. door)

Αυτή η ιεραρχία εξασφαλίζει ότι το σύστημα δεν κολλάει σε ατέρμονες προσπάθειες αλλά προσαρμόζεται δυναμικά.

## 6. Πειραματική Μελέτη - Στατικό Περιβάλλον

### 6.1 Μεθοδολογία

Εκτελέστηκαν **100 runs** στο στατικό περιβάλλον με τις εξής παραμέτρους:

- Οι θέσεις των στόχων (Table, Chair, Door) αλλάζουν τυχαία σε κάθε run
- Οι θέσεις των items (brush, color, key, code) παραμένουν σταθερές
- Οι πράκτορες ξεκινούν πάντα από τις ίδιες θέσεις: (1,1) και (3,3)
- Το run τερματίζεται όταν ολοκληρωθούν και οι 3 στόχοι

#### Μετρούμενες Μετρικές:

- Total Utility: Άθροισμα utilities των δύο πρακτόρων
- Total Steps: Συνολικά βήματα κίνησης
- Total Actions: Συνολικές ενέργειες (moves + pickups + paints + open)
- Agent-specific metrics: Ξεχωριστά για κάθε πράκτορα
- Success Rate: Ποσοστό επιτυχημένων runs

### 6.2 Συγκεντρωτικά Αποτελέσματα

| Μετρική              | Μέση Τιμή     | Τυπ. Απόκλιση | Min   | Max  |
|----------------------|---------------|---------------|-------|------|
| <b>Total Utility</b> | <b>1.4807</b> | 0.32          | 0.65  | 2.19 |
| Total Steps          | 39.5          | 12.4          | 23    | 67   |
| Total Actions        | 48.2          | 13.1          | 32    | 76   |
| Agent1 Utility       | 0.6224        | 0.45          | -0.82 | 1.62 |
| Agent2 Utility       | 0.8583        | 0.39          | -0.23 | 1.97 |
| Agent1 Steps         | 19.2          | 10.8          | 0     | 42   |
| Agent2 Steps         | 20.3          | 11.2          | 0     | 43   |
| <b>Success Rate</b>  | <b>100%</b>   | -             | -     | -    |



1. Όλα τα 100 runs ολοκληρώθηκαν επιτυχώς
2. **Υψηλό Total Utility:** Μέσο 1.48 (το μέγιστο θεωρητικό είναι 2.8)
3. **Χαμηλή Διασπορά:** Τυπική απόκλιση 0.32 δείχνει συνέπεια
4. **Ισορροπημένη Κατανομή:** Agent2 λαμβάνει υψηλότερο utility (0.86 vs 0.62)

## 6.3 Κατανομή Utilities

Η ανάλυση της κατανομής των utilities αποκαλύπτει:

### Distribution Ranges:

- **Excellent** (Utility > 1.8): 15% των runs
- **Very Good** (1.5 - 1.8): 22% των runs
- **Good** (1.2 - 1.5): 31% των runs
- **Adequate** (1.0 - 1.2): 20% των runs
- **Below Average** (< 1.0): 12% των runs

### Παρατηρήσεις:

- Το 68% των runs πετυχαίνει utility  $\geq 1.2$  (καλό επίπεδο)
- Τα "excellent" runs συμβαίνουν όταν οι στόχοι είναι κοντά μεταξύ τους και στους πράκτορες
- Τα "below average" runs οφείλονται σε δυσμενείς διατάξεις με μακρινούς στόχους

## 6.4 TOP 5 Runs - Ανάλυση Επιτυχίας

| Run | Total Utility | Steps | Agent1 Utility | Agent2 Utility | Αιτία Επιτυχίας                                |
|-----|---------------|-------|----------------|----------------|------------------------------------------------|
| 64  | 2.19          | 23    | 0.57           | 1.62           | Βέλτιστη διάταξη στόχων, μηδενικές συγκρούσεις |

|    |      |    |      |      |                                           |
|----|------|----|------|------|-------------------------------------------|
| 30 | 2.10 | 24 | 0.63 | 1.47 | Τέλεια παράλληλη εκτέλεση table/chair     |
| 23 | 2.05 | 25 | 0.53 | 1.52 | Αποδοτική κατανομή items από την αρχή     |
| 40 | 2.03 | 27 | 1.42 | 0.61 | Agent1 έλαβε πολλαπλές εργασίες αποδοτικά |
| 74 | 1.99 | 27 | 0.47 | 1.52 | Μηδενική αναμονή, smooth navigation       |

#### Κοινά Χαρακτηριστικά των TOP Runs:

1. **Βέλτιστη Διάταξη:** Οι στόχοι είναι τοποθετημένοι σε κοντινές αποστάσεις (cluster formation)
2. **Χωρικός Διαχωρισμός:** Οι πράκτορες δουλεύουν σε διαφορετικές περιοχές του πλέγματος
3. **Καθαρή Διαπραγμάτευση:** Κάθε πράκτορας παίρνει τις εργασίες που του ταιριάζουν χωρίς overlap
4. **Μηδενικά Move-Aside:** Δεν χρειάστηκε κανένα collision avoidance maneuver
5. **Γρήγορη Συλλογή Items:** Τα items ήταν στη διαδρομή προς τους στόχους

## 6.5 Κατανομή Εργασιών μεταξύ Πρακτόρων

Η ανάλυση των 100 runs αποκαλύπτει ενδιαφέροντα patterns στην κατανομή εργασιών:

| Εργασία | Agent1 (%) | Agent2 (%) | Μέση Utility Agent1 | Μέση Utility Agent2 |
|---------|------------|------------|---------------------|---------------------|
| Table   | 43%        | 57%        | 0.48                | 0.56                |
| Chair   | 41%        | 59%        | 0.46                | 0.54                |
| Door    | 52%        | 48%        | 0.42                | 0.38                |

Ερμηνεία:

- **Agent2 Dominance στο Painting:** Ο Agent2 αναλαμβάνει πιο συχνά table/chair (57-59%) επειδή συχνά βρίσκεται πιο κοντά στα painting items (brush στο 1,5 και color στο 5,5)
- **Agent1 Preference για Door:** Ο Agent1 αναλαμβάνει πιο συχνά το door (52%) επειδή ξεκινά από (1,1) κοντά στο key (1,4)
- **Φυσική Εξειδίκευση:** Το utility-based πρωτόκολλο οδηγεί σε φυσική εξειδίκευση χωρίς hardcoded rules

## 6.6 Συσχέτιση Utilities

Η στατιστική ανάλυση της συσχέτισης μεταξύ των utilities των δύο πρακτόρων αποκαλύπτει:

$\text{Correlation}(\text{Agent1\_Utility}, \text{Agent2\_Utility}) = -0.2187$

### Ερμηνεία της Αρνητικής Συσχέτισης:

Η **ασθενής αρνητική συσχέτιση** (-0.22) δείχνει ότι όταν ένας πράκτορας πετυχαίνει υψηλό utility, ο άλλος τείνει να έχει ελαφρώς χαμηλότερο. Αυτό είναι **θετικό σημάδι** και δείχνει:

1. **Εξειδίκευση:** Κάθε πράκτορας εστιάζει σε διαφορετικές εργασίες
2. **Αποφυγή Overlap:** Δεν ανταγωνίζονται για τις ίδιες εργασίες
3. **Συμπληρωματικότητα:** Ο ένας αναλαμβάνει εργασίες που ο άλλος δεν είναι βέλτιστος

### Παράδειγμα:

- Run με Agent1\_Utility=1.42 και Agent2\_Utility=0.61: Ο Agent1 πήρε 2 εργασίες (door + table), ο Agent2 μόνο chair
- Run με Agent1\_Utility=0.33 και Agent2\_Utility=1.41: Ο Agent2 πήρε table+chair, ο Agent1 μόνο door

## 6.7 Ανάλυση Βημάτων

### Κατανομή Steps:

- **Optimal** (23-30 steps): 23% των runs





- **Good** (31-40 steps): 35% των runs
- **Adequate** (41-50 steps): 30% των runs
- **Suboptimal** (>50 steps): 12% των runs

Το 58% των runs ολοκληρώνεται σε  $\leq 40$  βήματα, που είναι πολύ καλό για πολυπρακτορικό σύστημα.

#### Σύγκριση με Θεωρητικό Βέλτιστο:

- Θεωρητικό minimum (μονοπρακτορικό, ιδανική διάταξη): ~29 steps
- Μέσος αριθμός steps (πολυπρακτορικό): 39.5 steps
- Overhead: +36% λόγω coordination, collision avoidance, waiting

Το +36% overhead είναι **αποδεκτό** δεδομένης της πολυπλοκότητας του συντονισμού.

## 6.8 Collision Events

Κατά τη διάρκεια των 100 runs καταγράφηκαν:

| Event Type            | Συχνότητα | Μέσος Χρόνος Επίλυσης |
|-----------------------|-----------|-----------------------|
| Move-Aside Requests   | 47        | 1.2 seconds           |
| Oscillation Detection | 12        | 0.8 seconds           |
| Retreat Maneuvers     | 8         | 2.1 seconds           |
| Critical Recovery     | 0         | N/A                   |

#### Παρατηρήσεις:

1. **47% των runs** χρειάστηκαν τουλάχιστον ένα move-aside
2. Όλα επιλύθηκαν επιτυχώς χωρίς να οδηγήσουν σε deadlock
3. **Oscillation είναι σπάνιο** (12%) και διορθώνεται γρήγορα
4. **Μηδέν critical failures** δείχνει ισχυρό σύστημα

## 7. Δυναμικό Περιβάλλον

Στο δυναμικό περιβάλλον οι θέσεις των στόχων (Table, Chair, Door) **αλλάζουν κάθε 3 επιτυχημένες ενέργειες** κατά τη διάρκεια του run. Αυτό εισάγει σημαντική **αβεβαιότητα** και απαιτεί συνεχή **επαναπρογραμματισμό**.

### 7.1 Επίδραση της Δυναμικότητας

| Μετρική       | Στατικό | Δυναμικό | Αλλαγή               |
|---------------|---------|----------|----------------------|
| Total Utility | +1.48   | -0.76    | <b>-2.24</b> (-151%) |
| Total Steps   | 39.5    | 90.5     | <b>+51</b> (+129%)   |
| Success Rate  | 100%    | 95%      | <b>-5%</b>           |
| Deadlock Rate | 0%      | 5%       | <b>+5%</b>           |

#### Κύριες Επιπτώσεις:

1. **Δραματική Πτώση Utility:** Από +1.48 σε -0.76 λόγω αυξημένων βημάτων και επαναπρογραμματισμών
2. **Διπλάσια Βήματα:** Από 39.5 σε 90.5 βήματα κατά μέσο όρο
3. **Εμφάνιση Deadlocks:** 5% των runs τερματίζουν λόγω υπέρβασης ορίων

### 7.2 Μηχανισμός Replanning

Όταν ανιχνευτεί αλλαγή στο περιβάλλον (environment\_changed percept):

1. **Ακύρωση Τρέχουσας Πλοήγησης:** Το path που υπολογίστηκε δεν ισχύει πλέον
2. **Επαναυπολογισμός Utility:** Νέα απόσταση προς τον στόχο
3. **Επαναδιαπραγμάτευση** (αν χρειαστεί): Αν το utility έπεσε πολύ, μπορεί να παραχωρηθεί σε άλλον
4. **Νέο Path Planning:** Εκτέλεση A\* με νέα θέση στόχου

#### Κόστος Replanning:

- Χάνεται η πρόοδος προς τον παλιό στόχο
- Πιθανό backtracking προς τη νέα θέση

- Πιθανή σύγκρουση με άλλον πράκτορα που επίσης κάνει replanning

### 7.3 Worst Case Scenarios

Τα 5 χειρότερα runs στο δυναμικό περιβάλλον:

| Run | Utility | Steps |
|-----|---------|-------|
| 99  | -11.48  | 242   |
| 51  | -8.76   | 204   |
| 52  | -7.76   | 186   |
| 10  | -7.74   | 172   |
| 86  | -7.23   | 361   |

**Chase Pattern:** Ο χειρότερος τύπος αποτυχίας όπου ο πράκτορας "κυνηγεί" τον στόχο που συνεχώς μετακινείται μακριά.

### 7.4 Συσχέτιση στο Δυναμικό

$\text{Correlation}(\text{Agent1\_Utility}, \text{Agent2\_Utility}) = +0.5776$

Η **θετική συσχέτιση** (+0.58) στο δυναμικό είναι **αντίθετη** με το στατικό και δείχνει:

1. **Κοινά Μοτίβα Αποτυχίας:** Όταν ο ένας πάσχει από replanning, πάσχει και ο άλλος
2. **Συγχρονισμένα Προβλήματα:** Οι αλλαγές επηρεάζουν και τους δύο ταυτόχρονα
3. **Λιγότερη Εξειδίκευση:** Στο χάος της δυναμικότητας, η φυσική εξειδίκευση χάνεται

## 8. Περιπτώσεις Αποτυχίας του Πρωτοκόλλου

### 8.1 Circular Dependency Deadlock

Ένα από τα πιο ενδιαφέροντα σενάρια αποτυχίας είναι το circular dependency deadlock, που μπορεί να συμβεί όταν οι δύο πράκτορες κατέχουν συμπληρωματικά αντικείμενα για την ίδια εργασία. Για παράδειγμα, αν ο Agent\_1 έχει το key και

χρειάζεται το code για να ανοίξει την πόρτα, ενώ ταυτόχρονα ο Agent\_2 έχει το code και χρειάζεται το key για την ίδια εργασία, δημιουργείται μια κατάσταση αμοιβαίας εξάρτησης. Κατά τη διαπραγμάτευση, και οι δύο υπολογίζουν παρόμοιο utility για το door (0.8/απόσταση) και λόγω της ισοπαλίας ο Agent\_1 κερδίζει από το tie-breaking rule. Ωστόσο, ο Agent\_1 δεν μπορεί να ολοκληρώσει την εργασία χωρίς το code που κρατάει ο Agent\_2, ενώ ο Agent\_2 περιμένει επειδή έχασε τη διαπραγμάτευση. Αυτό το σενάριο είναι πολύ σπάνιο επειδή απαιτεί πολύ συγκεκριμένες συνθήκες: και οι δύο πράκτορες να έχουν συλλέξει ακριβώς ένα item για την ίδια εργασία πριν ξεκινήσει η διαπραγμάτευση. Το σύστημα το επιλύει μέσω του fail\_count mechanism - μετά από 3 αποτυχημένες προσπάθειες ο Agent\_1 κάνει drop το key, οπότε το utility του μηδενίζεται και επαναδιαπραγματεύονται για άλλες εργασίες.

## 8.2 Corridor Blocking

Το corridor blocking είναι μια πιο συχνή περίπτωση αποτυχίας που συμβαίνει όταν οι πράκτορες βρίσκονται σε στενό διάδρομο μεταξύ εμποδίων. Σε μια τυπική κατάσταση, ο Agent\_1 κινείται προς έναν στόχο και βρίσκει μπροστά του τον Agent\_2 σε στενό πέρασμα, περιορισμένο από obstacles στα πλάγια. Ο Agent\_1 στέλνει move-aside request, αλλά ο Agent\_2 δεν μπορεί να συμμορφωθεί επειδή και οι τέσσερις γειτονικές του θέσεις είναι είτε obstacles είτε εκτός ορίων του πλέγματος. Αυτό το πρόβλημα εμφανίζεται σε περίπου 3% των runs στο στατικό περιβάλλον και αυξάνεται στο 8% στο δυναμικό περιβάλλον όπου οι συχνές αλλαγές θέσεων των στόχων οδηγούν τους πράκτορες σε απρόβλεπτες διαδρομές. Το σύστημα το αντιμετωπίζει με την retreat strategy: ο Agent\_1 ανιχνεύει ότι ο Agent\_2 δεν κινήθηκε μετά από 1 δευτερόλεπτο, οπότε αποφασίζει να υποχωρήσει ο ίδιος 4 βήματα προς τα πίσω, περιμένει 1.5 δευτερόλεπτα για να δώσει χρόνο στον Agent\_2 να προχωρήσει, και μετά επιχειρεί ξανά την πλοήγηση. Αν και η retreat strategy προσθέτει overhead, αποτρέπει αποτελεσματικά τα deadlocks και επιτρέπει στο σύστημα να συνεχίσει.

## 8.3 Race Condition στα Αντικείμενα

Η race condition στα αντικείμενα συμβαίνει όταν και οι δύο πράκτορες ενδιαφέρονται για το ίδιο item χωρίς να το έχει κανένας στην κατοχή του κατά τη στιγμή της διαπραγμάτευσης. Για παράδειγμα, και οι δύο μπορεί να βλέπουν ότι το brush είναι διαθέσιμο στη θέση (1,5) και να υπολογίζουν utility για painting tasks που το απαιτούν. Ο Agent\_2 μπορεί να κερδίσει τη διαπραγμάτευση για το chair (utility 0.143) ενώ ο Agent\_1 να πάρει το table (utility 0.125). Και οι δύο ξεκινούν παράλληλα

προς το brush, αλλά ο Agent\_2 φτάνει πρώτος και το παίρνει. Όταν ο Agent\_1 φτάσει στη θέση και προσπαθήσει pickup, η ενέργεια αποτυγχάνει επειδή το item δεν είναι πια διαθέσιμο - ο Agent\_2 το έχει ήδη. Αν και φαίνεται προβληματικό, το σύστημα το διαχειρίζεται καλά: ο Agent\_1 ανιχνεύει αμέσως μέσω του other\_agent\_has(brush) check ότι δεν μπορεί να συνεχίσει την εργασία, το utility του για table πέφτει στο 0, και επαναδιαπραγματεύεται άμεσα για άλλη εργασία (συνήθως door). Δεν οδηγεί σε deadlock, απλά σε suboptimal allocation που διορθώνεται γρήγορα. Η αιτία αυτού του φαινομένου είναι ότι η διαπραγμάτευση βασίζεται σε εκτίμηση της μελλοντικής κατάστασης χωρίς reservation mechanism - δεν υπάρχει locking των items κατά τη διαπραγμάτευση.

## 8.4 Oscillation Loop

Το oscillation loop εμφανίζεται όταν ένας πράκτορας κινείται επανειλημμένα μεταξύ δύο θέσεων χωρίς να κάνει πρόοδο προς τον στόχο του. Ένα χαρακτηριστικό σενάριο περιλαμβάνει τον Agent\_2 στη θέση (5,2) που προσπαθεί να φτάσει στον στόχο (5,3), αλλά ο Agent\_1 βρίσκεται στο (5,3) εκτελώντας άλλη εργασία. Ο Agent\_2 δοκιμάζει να κινηθεί προς τα πάνω αλλά μπλοκάρεται, οπότε προσπαθεί να παρακάμψει κινούμενος κάτω στο (5,1), μετά δεξιά στο (4,1), αλλά συναντάει obstacle και επιστρέφει στο (5,1), μετά πάλι πάνω στο (5,2), και ο κύκλος επαναλαμβάνεται. Το σύστημα το ανιχνεύει μέσω του position history buffer που κρατάει τις τελευταίες 10 θέσεις - όταν αυτό το buffer περιέχει μόνο 2 μοναδικές θέσεις, σημαίνει ότι υπάρχει oscillation. Μετά από 2 cycles, ενεργοποιείται η pattern breaking strategy με asymmetric response: ο Agent\_2 (χαμηλότερη προτεραιότητα) εκτελεί active clear\_area και κινείται μακριά, ενώ ο Agent\_1 (υψηλότερη προτεραιότητα) μετακινείται σε safe zone και περιμένει. Αυτή η ασύμμετρη αντιμετώπιση είναι κρίσιμη για να αποφευχθεί η περίπτωση και οι δύο να ταλαντώνονται συγχρονισμένα.

## 8.5 Γενικές Παρατηρήσεις

Όλες οι παραπάνω περιπτώσεις αποτυχίας έχουν κοινά χαρακτηριστικά: ανιχνεύονται αξιόπιστα από το failure detection system, επιλύονται μέσω της ιεραρχίας recovery strategies (Simple → Moderate → Critical), και δεν οδηγούν σε permanent deadlock στο στατικό περιβάλλον (0% deadlock rate). Αυτό αποδεικνύει την ανθεκτικότητα της αρχιτεκτονικής που σχεδιάστηκε. Οι αποτυχίες είναι αναπόφευκτες σε κάθε πολυπρακτορικό σύστημα λόγω της ατελούς πληροφόρησης

(κάθε πράκτορας έχει τοπική όραση), της ασύγχρονης επικοινωνίας (το 500ms time window της διαπραγμάτευσης δημιουργεί καθυστέρηση), και της δυναμικότητας του περιβάλλοντος (αλλαγές μπορεί να συμβούν κατά την εκτέλεση). Το κλειδί για ένα επιτυχημένο σύστημα δεν είναι η πρόληψη όλων των αποτυχιών, κάτι που είναι πρακτικά αδύνατο, αλλά η γρήγορη και αποτελεσματική ανάκαμψη όταν αυτές συμβούν. Τα αποτελέσματα των πειραμάτων επιβεβαιώνουν ότι το σύστημα πετυχαίνει ακριβώς αυτό: detect fast, recover fast, continue execution.

## 9. Συμπεράσματα

Το πολυπρακτορικό σύστημα που αναπτύχθηκε πέτυχε τους στόχους της εργασίας με υψηλή αποτελεσματικότητα στο στατικό περιβάλλον. Η υλοποίηση του πρωτοκόλλου διαπραγμάτευσης βασισμένου σε χρησιμότητα (utility-based negotiation) επέτρεψε την αυτόνομη κατανομή εργασιών μεταξύ των δύο πρακτόρων, επιτυγχάνοντας 100% success rate στα 100 runs του στατικού περιβάλλοντος με μέση χρησιμότητα 1.48. Οι μηχανισμοί αποφυγής συγκρούσεων (move-aside protocol, retreat strategies, oscillation detection) λειτούργησαν άψογα, επιλύοντας όλες τις χωρικές συγκρούσεις χωρίς κανένα deadlock. Ένα σημαντικό εύρημα είναι η φυσική εξειδίκευση των πρακτόρων που αναδύθηκε χωρίς hardcoded rules: ο Agent\_2 ειδικεύτηκε σε painting tasks (57-59%) και ο Agent\_1 σε door tasks (52%), με την αρνητική συσχέτιση των utilities τους (-0.22) να επιβεβαιώνει τη συμπληρωματικότητα.

Η κύρια πρόκληση εντοπίστηκε στο δυναμικό περιβάλλον, όπου η συνεχής αλλαγή θέσεων των στόχων (κάθε 3 ενέργειες) οδήγησε σε δραματική πτώση της απόδοσης: το utility έπεσε από +1.48 σε -0.76 (πτώση 151%), τα βήματα διπλασιάστηκαν (90.5), και εμφανίστηκαν deadlocks σε 5% των runs. Αυτό δείχνει ότι το reactive replanning δεν επαρκεί για υψηλή δυναμικότητα και απαιτούνται προηγμένοι μηχανισμοί πρόβλεψης και συντονισμού. Παρόλα αυτά, το σύστημα αποδεικνύει επιτυχώς ότι utility-based negotiation σε συνδυασμό με reactive collision avoidance μπορεί να οδηγήσει σε αποτελεσματική συνεργασία χωρίς κεντρικό συντονιστή, προσφέροντας μια αποκεντρωμένη λύση κατάλληλη για εφαρμογές με σταθερό περιβάλλον όπου η ταχύτητα ολοκλήρωσης είναι κρίσιμη.